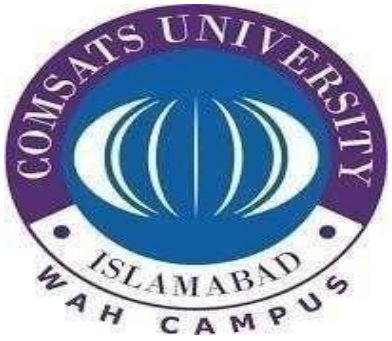
COMSATS UNIVERSITY ISLAMABAD WAH CAMPUS
SOFTWARE REQUIREMENT AND DESIGN SPECIFICATION



Supervisor Name:
Sir Ashfaq Ahmed

Submitted by:
Ayesha Nadeem (SP23-BSE -033)
Arooba Ahsan (SP23-BSE-042)

Section: BSE-7

Submission Date:
18-05-2026

Contents

CONTENTS **1**
1 INTRODUCTION **3**
1.1 SYSTEM INTRODUCTION 3
1.2 BACKGROUND OF THE SYSTEM 3
1.3 OBJECTIVES OF THE SYSTEM 3
1.4 SIGNIFICANCE OF THE SYSTEM 4
2 OVERALL DESCRIPTION **5**
2.1 PRODUCT PERSPECTIVE 5
2.2 PRODUCT SCOPE 5
2.3 PRODUCT FUNCTIONALITY 6
2.4 USERS AND CHARACTERISTICS 8
2.5 OPERATING ENVIRONMENT 9
3 SPECIFIC REQUIREMENTS **11**
3.1 FUNCTIONAL REQUIREMENTS 11
3.2 BEHAVIOR REQUIREMENTS 16
3.3 EXTERNAL INTERFACE REQUIREMENTS 22
4 OTHER NON-FUNCTIONAL REQUIREMENTS **30**
4.1 PERFORMANCE REQUIREMENTS 30
4.2 SAFETY AND SECURITY REQUIREMENTS 30
4.3 SOFTWARE QUALITY ATTRIBUTES 31
5 DESIGN DESCRIPTION **33**
5.1 COMPOSITE VIEWPOINT 33
5.2 LOGICAL VIEWPOINT 35
5.3 INFORMATION VIEWPOINT 36
5.4 INTERACTION VIEWPOINT 37
5.5 STATE DYNAMICS VIEWPOINT 41
5.6 ALGORITHM VIEWPOINT 41

Figures

Figure 2.1: System Context Diagram 5
Figure 3.2: Use case Diagram 17
Figure 3.3.1-A: Welcome/Splash Screen 22
Figure 3.3.1-B: Customer Dashboard 23
Figure 3.3.1-C: Cart & checkout Screen 24
Figure 3.3.1-D: Map Tracking 25
Figure 3.3.1-E: Vendor Dashboard 25
Figure 3.3.1-F: Rider Dashboard 26
Figure 3.3.1-G: Dispute Screen 26
Figure 3.3.1-H: Admin Dashboard 27
Figure 5.1.1: Deployment Diagram 33
Figure 5.1.2: Package Diagram 34
Figure 5.2: Class Diagram 35
Figure 5.3: Entity Relationship Diagram 36
Figure 5.4.1: Customer-View Sequence Diagram 37
Figure 5.4.2: Vendor-View Sequence Diagram 38
Figure 5.4.3: Rider-View Sequence Diagram 39
Figure 5.4.4: Admin-View Sequence Diagram 40
Figure 5.5: State Machine Diagram 41

Tables

Table 3.1.1: User Management & Authentication 11
Table 3.1.2: Order Management & Processing 11
Table 3.1.3: Real-Time Tracking & Delivery Verification 12
Table 3.1.4: Vendor Management & Product System 13
Table 3.1.5: Rider Management & Assignment 13
Table 3.1.6: Payment Integration & Transaction Management 14
Table 3.1.7: Feedback, Rating & Dispute Resolution 14
Table 3.1.8: Admin Dashboard & Analytics 15
Table 3.1.9: Notification Management 15
Table 3.1.10: Security & Data Protection 16
Table 3.2.1: Register/Login Usecase 18
Table 3.2.2: Browse & Order Products Usecase 18
Table 3.2.3: Vendor Product Management Usecase 18
Table 3.2.4: Order Assignment & Delivery Usecase 19
Table 3.2.5: Real Time Order Tracking Usecase 19
Table 3.2.6: Payment & Refund Processing Usecase 20
Table 3.2.7: Feedback & dispute Management Usecase 20
Table 3.2.8: Admin Monitoring & Analytics Usecase 21
Table 3.3.3: Software Interfaces 28
Table 5.6: Algorithm Viewpoint 41

1 Introduction

1.1 System Introduction

Tez Rider is a verified, multi-vendor delivery platform built with Flutter (Android/iOS front-end) and Supabase (PostgreSQL back-end). It connects Customers, Vendors, Riders, and Administrators on a unified platform to enable product browsing across groceries, medicines, food, and daily essentials; real-time GPS-tracked delivery; secure payment; and a structured pipeline. It also opens secure delivery channels for overseas Pakistanis sending essentials home.

1.2 Background of the System

Pakistan's e-commerce market is dominated by cash-on-delivery transactions and phone-based ordering. Existing platforms such as Foodpanda target food exclusively and lack the verification depth for a general goods marketplace. Most local solutions use Firebase (NoSQL), which sacrifices relational data integrity and complicates financial reporting and audit trails. Tez Rider differentiates itself by adopting Supabase (PostgreSQL) as the back-end providing full SQL query capability, Row-Level Security (RLS) policies for per-user data isolation, real-time change-data capture, and built-in authentication within a single managed service. The Flutter front-end delivers a consistent Android/iOS experience from one codebase, while the REST + Realtime API layer supports future web and third-party integrations.

1.3 Objectives of the System

- Build a Flutter mobile application serving four distinct roles: Customer, Vendor, Rider, and Administrator, across groceries, medicines, food and daily essentials.
- Implement a Supabase (PostgreSQL + RLS) back-end for all data persistence, authentication, real-time GPS updates, and file storage.
- Integrate Maps API for live GPS tracking of Rider location, route

calculation, and ETA estimation.
- Enforce CNIC-based Rider verification.
- Build a mandatory delivery checklist system to reduce incorrect and damaged order incidents.
- Provide a multi-tier dispute-resolution module with evidence upload, Admin dispute handling and automated refund triggering.
- Deliver an Admin analytics dashboard with real-time order monitoring, revenue reports, and rider/vendor performance KPIs.

1.4 Significance of the System

Tez Rider carries strategic significance across multiple domains:

- Empowers small and medium vendors including home-based businesses to sell and deliver products digitally without independent logistics infrastructure.
- Gives Riders an automated, GPS-verified, performance-tracked job system with transparent earnings and protection against unfair dispute resolution.
- The Admin analytics module supports data-driven decisions on vendor approval, rider performance, and platform growth.

2 Overall Description

2.1 Product Perspective

Tez Rider is a self-contained, cloud-native platform operating on a client–server model. The Flutter mobile application communicates with the Supabase back-end over HTTPS/REST for standard operations and WebSocket (wss://) for real-time GPS and order-status streams. The Supabase instance hosts the PostgreSQL relational database, GoTrue authentication service, Supabase Storage (product images, delivery proof photos), Realtime channels, and the PostgREST API layer with RLS-enforced policies.
External services like Maps API are integrated either directly from the Flutter client or via Supabase Edge Functions.

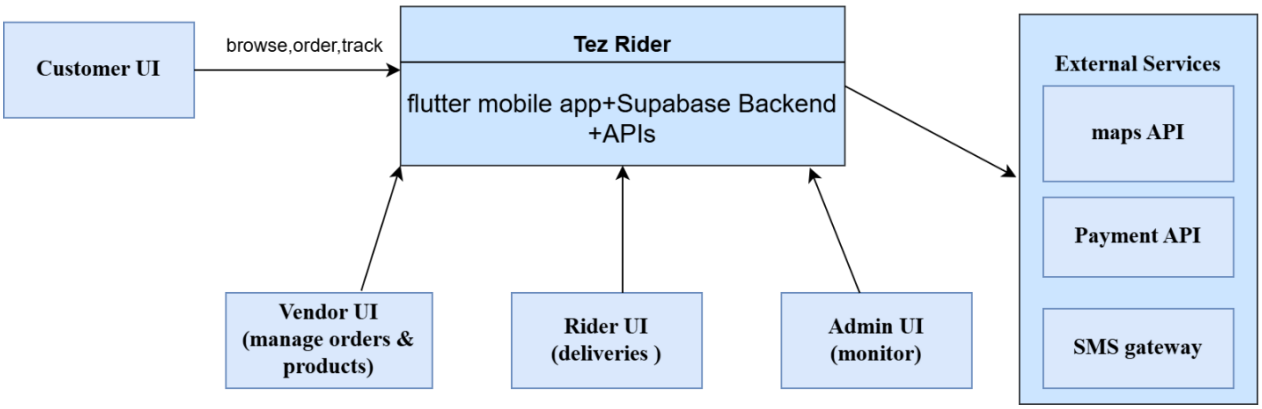


Figure 2.1: System Context Diagram

2.2 Product Scope

Tez Rider is a multi-vendor delivery management system that handles the complete order lifecycle from customer registration to final delivery. The system allows users to browse products, manage cart and place orders. After order placement, vendors process requests and CNIC-verified riders are assigned for delivery. The system provides real-time GPS to ensure secure and transparent operations.
The system also supports structured onboarding for vendors and riders, where vendors are verified before listing products and riders are registered using CNIC verification.

Out of scope for the current version: web-browser customer interface, subscription/recurring orders, and multi-currency support. These may be addressed in a future release.

2.3 Product Functionality

The Tez Rider system is organized into ten functional modules as defined in the approved FYP proposal:

2.3.1 User Management & Authentication System

- User registration for all four roles (Customer, Vendor, Rider, Admin) with JWT token-based authentication via Supabase Auth.
- Role-based access control (RBAC) enforced at both the Flutter navigation layer and the Supabase RLS database layer.
- User profile management: name, phone, email, address, avatar, and role-specific fields.

2.3.2 Order Management & Processing System

- Multi-category product catalogue browsing (groceries, medicines, food, daily essentials) with search and category filters.
- Persistent shopping cart with multi-vendor support — customers can add items from multiple vendors in one cart.
- Order placement with full status tracking: PENDING → ACCEPTED → IN_TRANSIT → DELIVERED / CANCELLED.
- Order history with per-order itemized breakdown and status timeline.

2.3.3 Real-Time Tracking & Delivery Verification System

- Live GPS tracking of Rider location streamed via Supabase Realtime every 2-3 seconds; rendered on a Maps widget with route polyline and ETA.
- Mandatory pre-delivery checklist for Riders to reduce incorrect or damaged order incidents.

- Mandatory pre-delivery checklist for Riders to reduce incorrect or damaged order incidents.

2.3.4 Vendor Management & Product System

- Vendor registration with business name and address (subject to Admin approval).
- Full product CRUD: add/edit/delete listings with name, description, price, category, images, and stock quantity stored in Supabase Storage.
- Inventory management with configurable low-stock alerts.
- Order accept/reject workflow with configurable timeout; auto-cancellation on timeout.
- Vendor earnings dashboard with per-order transaction breakdown and aggregate performance analytics.

2.3.5 Rider Management & Assignment System

- Rider registration with CNIC verification for identity validation.
- Availability toggle (online/offline) and location-based intelligent assignment algorithm via Supabase Edge Function (nearest available verified rider).
- Delivery performance tracking: completion rate, average delivery time, customer rating.
- Rider earnings history with per-delivery breakdown.

2.3.6 Feedback, Rating & Dispute Resolution System

- Dual feedback mechanism: public star ratings (1–5) for Rider and Vendor displayed on their profiles, and private complaint submission.
- Structured dispute resolution workflow: Customer raises dispute within 24 hours with evidence images → Admin reviews → RESOLVED

(refund) or REJECTED.
- Automated blocking of consistently low-rated vendors or riders after configurable threshold breaches.
- Running average rating computation updated in real time on Vendor/Rider profiles.

2.3.7 Admin Dashboard & Analytics System

- Vendor and Rider approval/rejection workflows with document review.
- Real-time order monitoring dashboard with live counts and map view.
- Analytics charts: daily/weekly order volumes, revenue trends, dispute rates, top-performing vendors, rider KPIs.
- Business intelligence reports exportable for management review.
- User management: block/unblock users; resolve escalated disputes.

2.3.8 Notification & Communication System

- Multi-channel push notifications for order status changes, rider assignment, dispute updates, and Admin broadcasts.
- SMS OTP delivery via SMS Gateway for delivery confirmation.
- Notifications for order confirmations, invoices, and account actions.

2.3.9 Security & Data Protection System

- Supabase RLS policies enforcing per-user data isolation at the PostgreSQL layer.
- Input validation and SQL injection prevention on all API endpoints.
- Audit logging of all critical operations in a security_audit_log table.

2.4 Users and Characteristics

2.4.1 Primary Users

- **Customers** are everyday mobile users in urban Pakistan, aged 16–50, with moderate smartphone proficiency. They require a simple, fast UI with support for local payment methods, Urdu-friendly navigation, and full delivery visibility including rider identity and live GPS.
- **Vendors** are small-to-medium business owners including home-based sellers with basic smartphone literacy. They need a straightforward dashboard for inventory management, order processing, and earnings tracking without

requiring technical knowledge.

- **Riders** are gig workers operating in variable-connectivity environments. They need a lightweight, battery-efficient interface with clear job queues, turn-by-turn navigation, and simple proof-capture tools. CNIC registration is mandatory.
- **Administrators** are internal platform operators with higher technical proficiency. They interact with the platform infrequently but require comprehensive moderation tools, real-time monitoring, and analytical reporting.

2.4.2 Secondary Users

- This app doesn't contain any secondary user or supporting roles.

2.5 Operating Environment

2.5.1 Hardware Requirements

- **For Customers**
  • Smartphone (Android/iOS) required
  • Processor: Quad-core 1.8 GHz or higher (ARM or x86_64)
  • RAM: Minimum 3 GB (recommended 6 GB+)
  • Storage: 150 MB app installation; additional 500 MB for cached data and media
  • Network: Minimum 2 Mbps internet; 4G/LTE recommended for smooth usage
  • GPS: Required for real-time delivery tracking
  • Camera: Optional (used for profile and support features)

- **For Vendors**
  • Smartphone (Android/iOS) required
  • Processor: Dual-core or higher
  • RAM: Minimum 3 GB
  • Storage: 150 MB app space
  • Network: Stable internet required
  • Camera: Optional (product images)

- **For Riders**
  • Smartphone (Android/iOS) required
  • Processor: Quad-core 1.8 GHz or higher
  • RAM: 4 GB minimum recommended

• Storage: 150 MB + 500 MB for media
• Network: Stable 4G/LTE required
• GPS: Mandatory
• Camera: Required

- **For Admins**
  • Laptop/PC or high-performance device preferred
  • Processor: Quad-core or higher
  • RAM: 4 GB minimum
  • Storage: 200 MB+
  • Network: High-speed internet recommended

2.5.2 Software Environment

- **Front-End Requirements**
- Framework: Flutter
- Maps & Location Services: flutter_map ^6.1.0, geolocator ^12.0.0
- Authentication & Security: flutter_secure_storage ^9.0.0, local_auth ^2.3.0

- **Back-End Requirements**
- Backend Platform: Supabase
- Database: PostgreSQL 15
- Backend Services: Supabase Authentication, Supabase Realtime, Supabase Storage, Supabase Edge Functions
- Notifications Service: SMS Gateway REST API
- Payment Integration: Third-party Payment API

- **Operating System Requirements**
- Android: Android 8.0 (API 26) +
- iOS: iOS 13.0+
- Flutter SDK: Flutter 3.x
- Dart SDK: Dart 3.x

3 Specific Requirements

3.1 Functional Requirements

3.1.1 User Management & Authentication

Table 3.1.1: User Management & Authentication

Functional Req ID#	Requirements
FR-01	The system shall allow users to register using name, email, phone number, password, and role selection.
FR-02	The system shall authenticate users using Supabase authentication and generate JWT tokens.
FR-03	The system shall keep Vendor and Rider accounts inactive until Admin approval is completed.
FR-04	The system shall allow Riders to upload CNIC number.
FR-05	The system must enforce role-based access control for all system users.

3.1.2 Order Management & Processing

Table 3.1.2: Order Management & Processing

Functional Req ID#	Requirements

FR-06	The system shall allow customers to browse and search products by category and keyword.
FR-07	The system shall allow customers to add products to a persistent shopping cart.
FR-08	The system shall create order, order-item, and payment records during checkout.
FR-09	The system shall allow Vendors to accept or reject customer orders.
FR-10	The system shall automatically cancel timed-out orders and trigger refunds.
FR-11	The system shall display the complete order status timeline to customers.
FR-12	The system shall maintain and display itemized order history records.

3.1.3 Real-Time Tracking & Delivery Verification

Table 3.1.3: Real-Time Tracking & Delivery Verification

Functional Req ID#	Requirements
FR-13	The system shall stream Rider GPS coordinates every 2–3 seconds.
FR-14	The system shall provide live Rider tracking with route and ETA information**.**
FR-15	The system shall display rider info before delivery.

3.1.4 Vendor Management & Product System

Table 3.1.4: Vendor Management & Product System

Functional Req ID#	Requirements
FR-16	The system shall allow Vendors to create, update, and delete product listings.
FR-17	The system shall automatically update stock quantities after order placement or cancellation.
FR-18	The system shall provide Vendors with an earnings and performance dashboard.

3.1.5 Rider Management & Assignment

Table 3.1.5: Rider Management & Assignment

Functional Req ID#	Requirements
FR-19	The system shall assign the nearest available verified Rider to delivery orders.
FR-20	The system shall expand the Rider search radius and notify Admin if no Rider is available within 5 km.
FR-21	The system shall update Rider online/offline availability status in real time.

FR-22	The system shall calculate and store Rider performance metrics after each delivery.

3.1.6 Feedback, Rating & Dispute Resolution

Table 3.1.6: Feedback, Rating & Dispute Resolution

Functional Req ID#	Requirements
FR-23	The system shall allow customers to submit ratings and feedback after delivery.
FR-24	The system shall calculate and display updated average ratings in real time.

FR-25	The system shall allow customers to raise disputes with evidence images.
FR-26	The system shall allow Admins to resolve or reject customer disputes.

3.1.7 Admin Dashboard & Analytics

Table 3.1.7: Admin Dashboard & Analytics

Functional Req ID#	Requirements
FR-27	The system shall allow Admins to approve or reject Vendor and Rider registrations.
FR-28	The system shall allow Admins to block or unblock users.
FR-29	The system shall provide real-time analytics and monitoring dashboards for Admins.
FR-30	The system shall allow Admins to send notifications to users.

3.1.8 Notification Management

Table 3.1.8: Notification Management

Functional Req ID#	Requirements
FR-31	The system shall send order and system notifications to users in real time.

FR-32	The system shall send email notifications for invoices, confirmations, and account actions.
FR-33	The system shall deliver OTP messages to customers through the SMS gateway.

3.1.9 Security & Data Protection

Table 3.1.10: Security & Data Protection

Functional Req ID#	Requirements
FR-42	The system must enforce Row-Level Security (RLS) policies for database access.
FR-43	The system shall validate and sanitize all user inputs before processing.
FR-44	The system shall maintain audit logs for all critical Admin actions.

3.2 Behavior Requirements

The Use Case Diagram below captures all actors (Customer, Vendor, Rider, Admin) and their interactions within the Tez Rider system boundary. Key include/extend relationships reflect mandatory and optional extensions of the primary use cases.

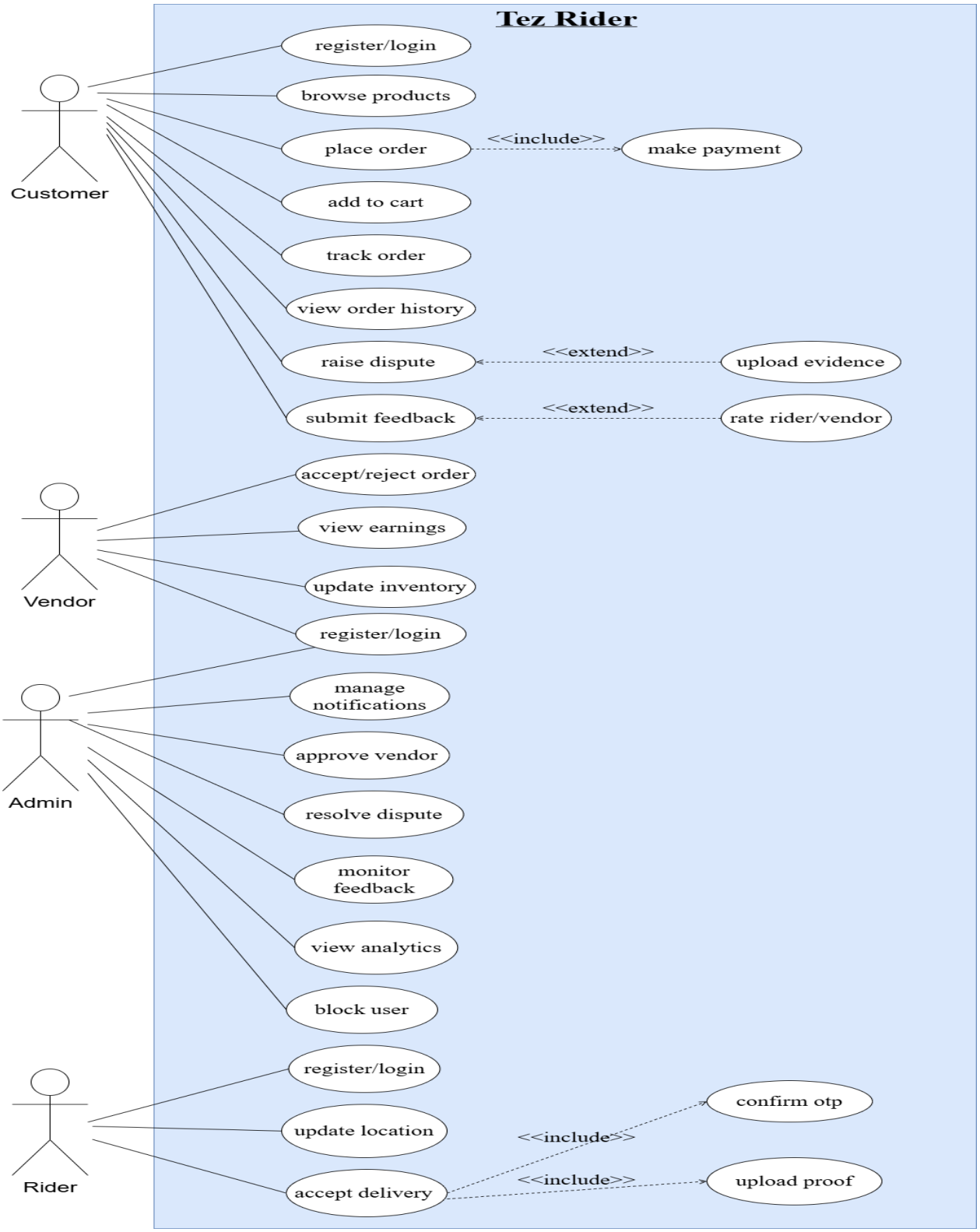


Figure 3.2: Use case Diagram

UC-01: Register / Login:

Table 3.2.11: Register/Login Usecase

Field	Description
Use Case ID	UC-01
Use Case Name	Register / Login
Actor(s)	Customer, Vendor, Rider, Admin
Description	Allows users to create accounts and log in to the system securely.
Pre-condition	User has internet access and valid credentials.
Main Flow	1. User opens login/register page. 2. User enters required information. 3. System validates data. 4. Account is created or user authenticated. 5. JWT session token generated. 6. User redirected to role-specific dashboard.
Post-condition	User is authenticated and active session is created.

UC-02: Browse & Order Products:

Table 3.2.12: Browse & Order Products Usecase

Field	Description
Use Case ID	UC-02
Use Case Name	Browse & Order Products
Actor(s)	Customer
Description	Customer browses products and places an order.
Pre-condition	Customer is logged in and products are available.
Main Flow	1. Customer browses categories. 2. Customer searches/selects products. 3. Products added to cart. 4. Customer proceeds to checkout. 5. Payment completed. 6. Order created successfully.
Post-condition	Order is stored and visible in order history.

UC-03: Vendor Product Management:

Table 3.2.13: Vendor Product Management Usecase

Field	Description

Use Case ID	UC-03
Use Case Name	Vendor Product Management
Actor(s)	Vendor
Description	Vendor manages product listings and stock.
Pre-condition	Vendor account approved and logged in.
Main Flow	1. Vendor opens dashboard. 2. Vendor adds/updates/deletes products. 3. Vendor updates stock quantity and pricing. 4. System saves product information.
Post-condition	Product catalog updated successfully.

UC-04: Order Assignment & Delivery:

Table 3.2.14: Order Assignment & Delivery Usecase

Field	Description
Use Case ID	UC-04
Use Case Name	Order Assignment and Delivery
Actor(s)	Rider and System
Description	System assigns orders to riders for delivery.
Pre-condition	Order confirmed and rider available.
Main Flow	1. System detects nearest rider. 2. Rider receives assignment request. 3. Rider accepts order. 4. Rider picks up order. 5. GPS tracking starts. 6. Rider delivers order.
Post-condition	Order marked as delivered successfully.

UC-05: Real-Time Order Tracking:

Table 3.2.15: Real Time Order Tracking Usecase

Field	Description
Use Case ID	UC-05
Use Case Name	Real-Time Order Tracking
Actor(s)	Customer, Rider

Description	Customer tracks rider location in real time.
Pre-condition	Order assigned to Rider.
Main Flow	1. Rider GPS coordinates streamed. 2. System updates route and ETA. 3. Customer views live rider location on map.
Post-condition	Customer receives real-time tracking updates.

UC-06: Payment & Refund Processing:

Table 3.2.16: Payment & Refund Processing Usecase

Field	Description
Use Case ID	UC-06
Use Case Name	Payment & Refund Processing
Actor(s)	Customer, System
Description	Handles secure payments and refunds.
Pre-condition	Customer proceeds to checkout.
Main Flow	1. Customer selects payment method. 2. System processes transaction. 3. Payment validated. 4. Invoice generated.
Post-condition	Payment/refund recorded successfully.

UC-07: Feedback & Dispute Management:

Table 3.2.17: Feedback & dispute Management Usecase

Field	Description
Use Case ID	UC-07
Use Case Name	Feedback and dispute Management
Actor(s)	Customer, Admin
Description	Customer submits ratings or raises disputes.
Pre-condition	Order completed successfully.
Main Flow	1. Customer submits rating and feedback. 2. Customer uploads dispute evidence if required. 3. Admin reviews dispute. 4. Admin resolves or rejects dispute.
Post-condition	Feedback/dispute stored and processed.

UC-08: Admin Monitoring & Analytics:

Table 3.2.18: Admin Monitoring & Analytics Usecase

Field	Description
Use Case ID	UC-08
Use Case Name	Admin Monitoring & Analytics
Actor(s)	Admin
Description	Admin monitors system activities and analytics.
Pre-condition	Admin logged in.
Main Flow	1. Admin opens analytics dashboard. 2. Admin reviews orders, users, and reports. 3. Admin approves/rejects vendors and riders. 4. Admin sends notifications or blocks users.
Post-condition	System administration actions completed successfully.

3.3 External Interface Requirements

3.3.1 User Interfaces

The Flutter application provides the following primary screens per role:

- **Welcome/Splash Screen:** Displays the TEZRIDER logo with the tagline "Delivering Excellence, Every Time", animated delivery-themed background, onboarding indicators, and introductory interface for users before accessing the application.

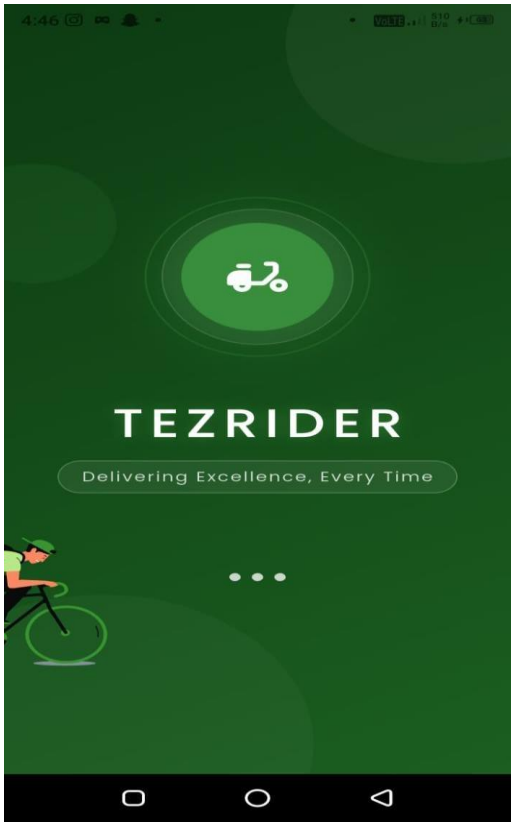
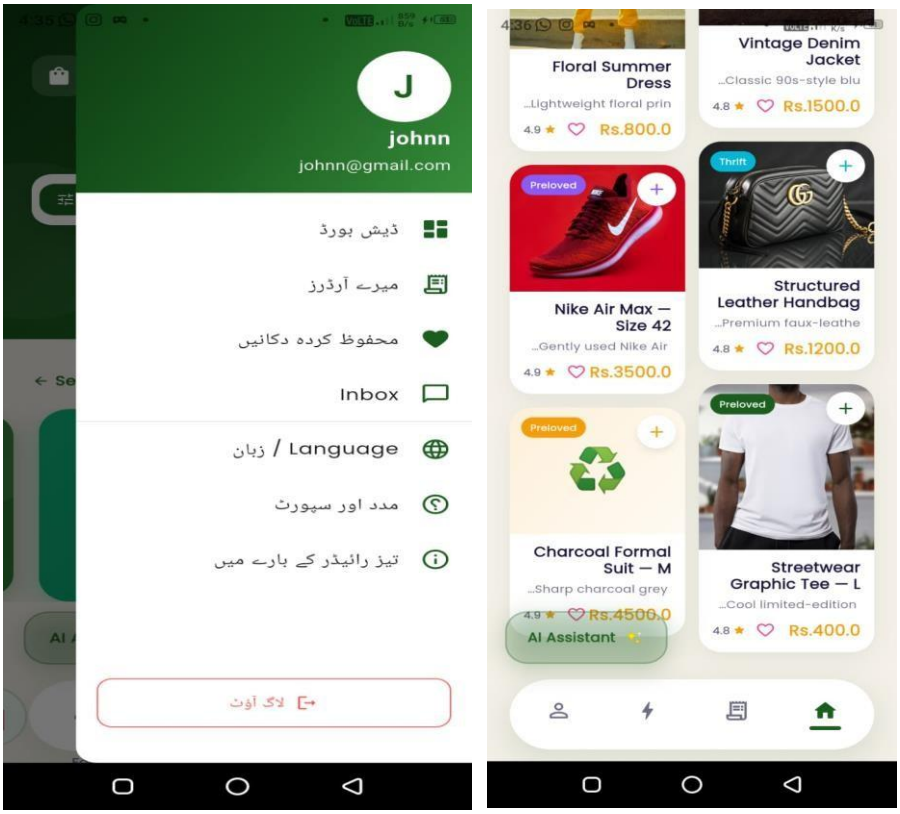


Figure 3.3.1-A: Welcome/Splash Screen

- **Customer Home / Catalogue**: Displays personalized greeting, search bar, featured products section, category filters, promotional banners, and bottom navigation for easy browsing and ordering.



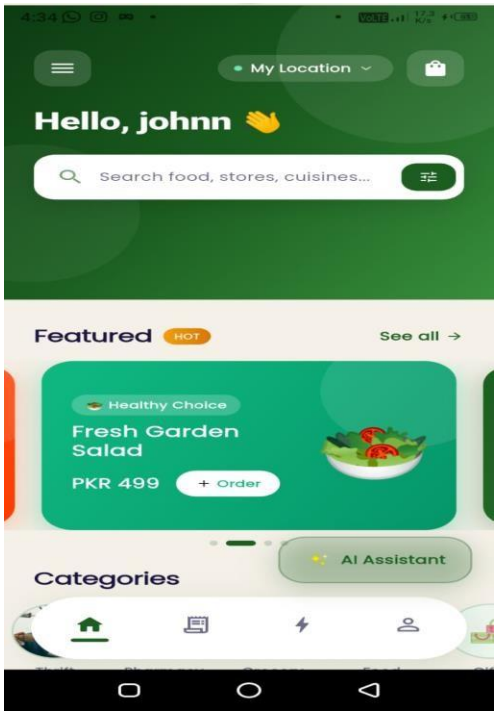


Figure 3.3.1-B: Customer Dashboard

- **Cart & Checkout**: Line items grouped by vendor, quantity steppers, order total, payment method selector.

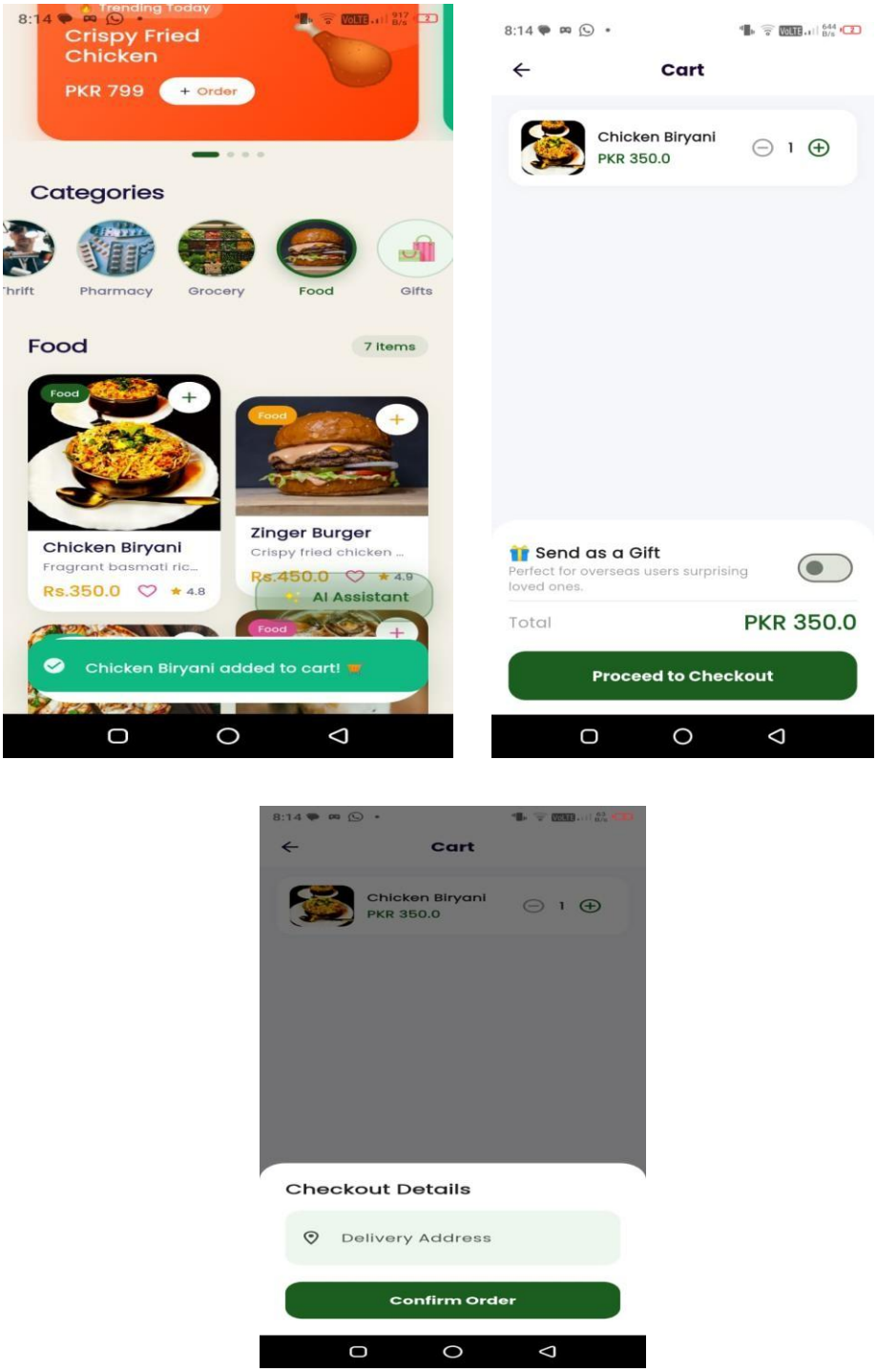


Figure 3.3.1-C: Cart & checkout Screen

- **Order Tracking:** Maps widget with live Rider pin, route polyline, ETA chip, Rider identity card (photo, rating, masked CNIC), order status stepper.

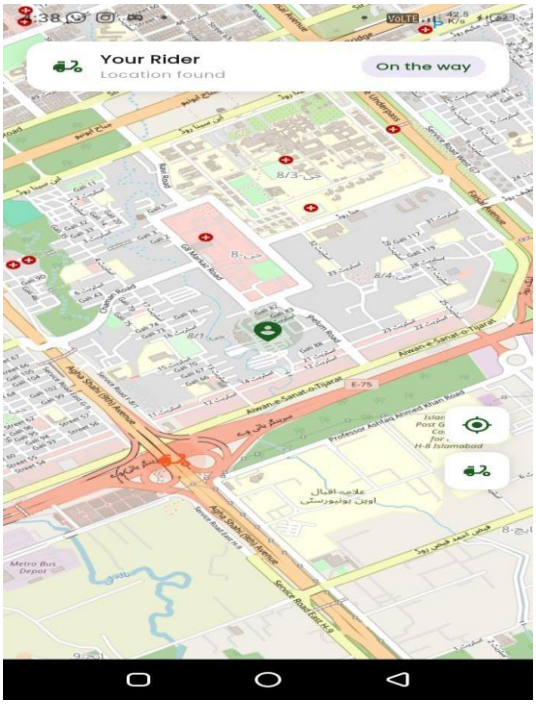


Figure 3.3.1-D: Map Tracking

- **Vendor Dashboard:** Pending/Accepted order queue tabs, inventory list with stock indicators, earnings summary card, analytics chart.

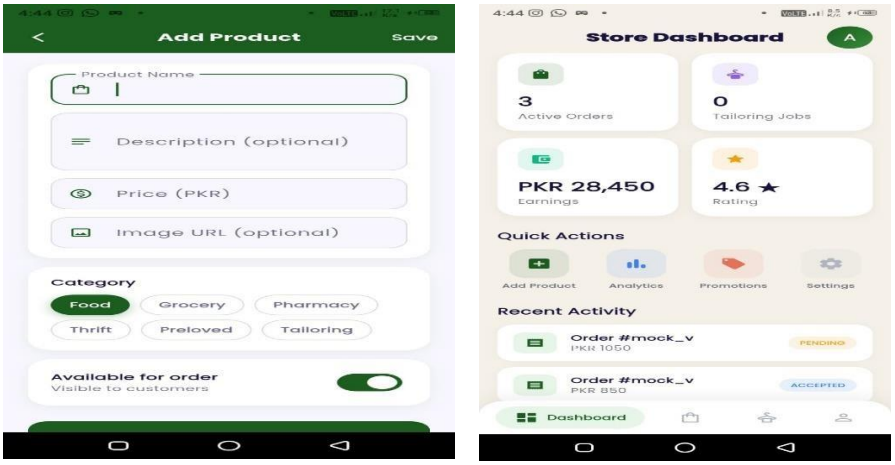


Figure 3.3.1-E: Vendor Dashboard

- **Rider Dashboard:** Assigned job card with pickup and delivery addresses, Maps navigation, pre-delivery checklist, OTP entry field, proof-of-delivery photo upload.

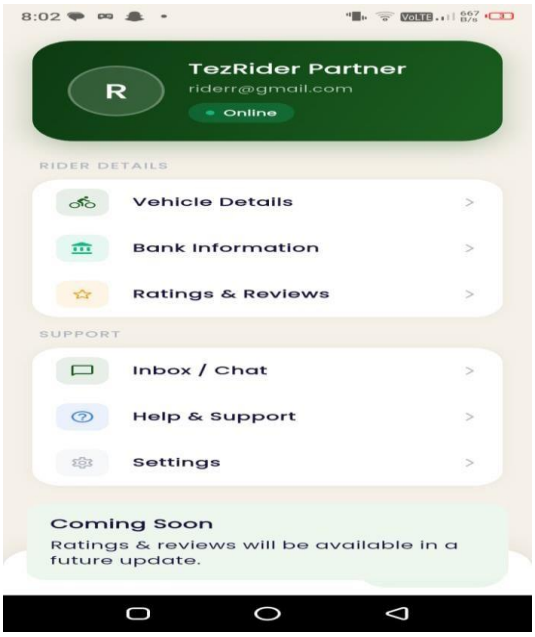


Figure 3.3.1-F: Rider Dashboard

- **Dispute Screen:** Allows users to select order reference, choose dispute reason from dropdown menu, upload supporting evidence images, view dispute status, and submit complaints within the specified resolution time window.

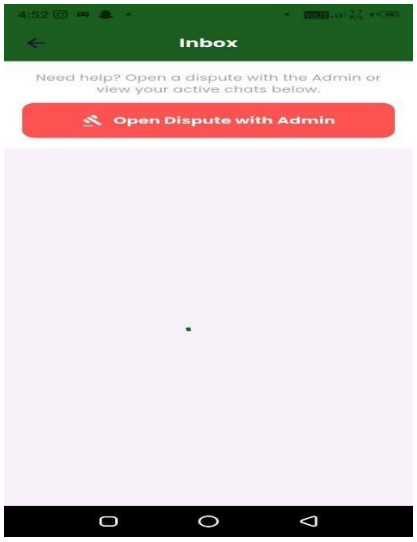


Figure 3.3.1-G: Dispute Screen

- **Admin Panel:** Vendor/Rider approval queues with document viewer, user management table, dispute queue, real-time analytics charts, broadcast notification composer.

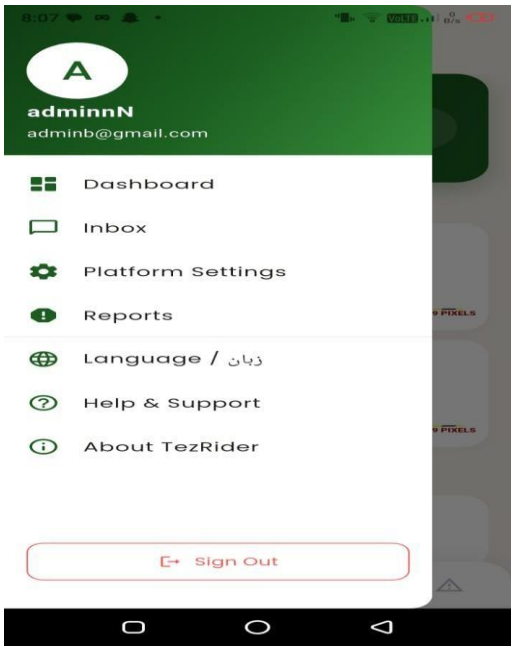


Figure 3.3.1-H: Admin Dashboard

3.3.2 Hardware Interfaces

The Tez Rider system interfaces with various hardware components of Android and iOS mobile devices, including the touchscreen display for user interaction, GPS module for real-time rider tracking and location services, camera for CNIC scanning and proof-of-delivery image capture, local storage for caching session and application data, and network hardware (Wi-Fi/3G/4G/5G) for communication with Supabase services, payment gateways, Maps APIs and SMS gateways. The system uses Flutter libraries such as geolocator for GPS access and image_picker for camera integration.

3.3.3 Software Interfaces

Table 3.3.3: Software Interfaces

Component	Technology	Purpose	Protocol / Format
Database	Supabase PostgreSQL 15	Relational storage for all entities; RLS for per-user data isolation	PostgREST REST API +supabase_flutter SDK
Authentication	Supabase GoTrue	JWT issuance, session refresh, email & phone OTP	HTTPS REST; JWT
Real-Time	Supabase Realtime	Rider GPS broadcast; order-status live updates	WebSocket Secure (wss://)
File Storage	Supabase Storage	Product images and CNIC docs,	HTTPS multipart upload;
Server Logic	Supabase Edge Functions	Payment validation, rider assignment, SMS dispatch	HTTPS
Maps	Maps SDK + Directions API	Live map rendering, ETA	HTTPS REST + Flutter native SDK
Payments	Payment API	Payment initiation, refunds	HTTPS REST;

SMS OTP	SMS Gateway	OTP for delivery confirmation	HTTPS REST

3.3.4 Communications Interfaces

All Flutter-to-Supabase communication uses HTTPS/TLS for REST calls and WebSocket Secure (wss://) for Realtime GPS and order-status subscriptions, with JWT-based channel authorization. Payment gateway arrive as HTTPS POST requests; signature validation is enforced in a Supabase Edge Function before any database write. OTP SMS messages are dispatched from Edge Functions to the chosen SMS gateway over HTTPS REST.

4 Other Non-functional Requirements

4.1 Performance Requirements

The Tez Rider system shall provide fast and reliable performance with quick API responses, smooth screen loading, real-time GPS tracking, efficient payment processing, timely OTP delivery, and stable operation for multiple concurrent users without significant performance degradation.

4.2 Safety and Security Requirements

4.2.1 Safety Requirements

- **S-01:** The system shall require a confirmation dialogue before any destructive action (order cancellation, account deletion, block user).
- **S-02**: If the Rider's GPS stream is interrupted for more than 30 seconds, the Customer's map shall display a 'Location temporarily unavailable' message rather than showing a stale pin.
- **S-03:** Payment failures shall trigger an immediate rollback of order status to PENDING and notify the Customer within 2 seconds, with a clear error message and retry option.
- **S-04:** OTP entry shall allow a maximum of 3 attempts; on failure the OTP expires and a fresh one must be requested, preventing brute-force delivery confirmation.
- **S-05:** Files uploaded to Supabase Storage shall pass validation (images only for proof and CNIC) before being accepted.
- **S-06:** Vendor order acceptance timeout (2-3 minutes default) shall trigger automated order cancellation and customer notification to prevent indefinitely pending orders.

4.2.2 Security Requirements

- **SEC-01:** All Supabase database tables shall have Row-Level Security (RLS) policies ensuring each user can only access records they own or are authorized to access by their role.
- **SEC-02:** User passwords shall be hashed by Supabase GoTrue; plain-text passwords shall never be stored or logged.
- **SEC-03:** JWT access tokens shall expire after 1 hour; refresh tokens after

7 days. Silent refresh is managed by the supabase_flutter SDK.

- **SEC-04:** All API keys (Supabase anon key, Maps key, payment credentials) shall be stored as Flutter --dart-define environment variables or Supabase Vault secrets; never hard-coded or committed to version control.
- **SEC-05:** Payment gateway callbacks shall be authenticated via signature validation before any order-state mutation.
- **SEC-06:** Rate limiting (max 100 requests/minute per authenticated user) shall be enforced at the Supabase API Gateway layer.
- **SEC-07:** All critical Admin actions shall be written to security_audit_log with actor_id, action, target_type, target_id, and created_at.
- **SEC-08:** Supabase Storage media files shall be served exclusively via time-limited signed URLs (15-minute TTL) to prevent hotlinking and unauthorised access.
- **SEC-09:** CNIC data shall be stored encrypted and displayed in masked form (3***0-*******-*) at all customer-facing endpoints.
- **SEC-10:** All user-supplied inputs shall be validated and sanitised in Supabase Edge Functions to prevent SQL injection, XSS, and malformed-request attacks.

4.3 Software Quality Attributes

4.3.1 Reliability

Supabase managed infrastructure provides a 99.9% uptime SLA with automatic failover. The Flutter app implements optimistic UI updates with automatic retry on network failure (exponential back-off), ensuring graceful degradation in low-connectivity environments common in Pakistan.

4.3.2 Usability

Onboarding flows are designed to achieve Time-to-First-Order under 3 minutes for new Customers. Rider-facing screens prioritize large buttons and minimal text to support variable literacy levels.

4.3.3 Maintainability

The Flutter codebase follows Clean Architecture (presentation / domain / data layers)

with feature-based folder structure. Supabase database schema changes are managed exclusively through versioned migration files; no direct production schema modifications are permitted. All business logic resides in Riverpod providers (testable without UI) and Supabase Edge Functions.

4.3.4 Portability

Flutter's single codebase targets Android and iOS without platform-specific modifications. Supabase can be self-hosted or migrated to any cloud provider (AWS, GCP, Azure) by updating only two environment variables (SUPABASE_URL, SUPABASE_ANON_KEY). No vendor lock-in exists at the data layer since PostgreSQL is a fully open standard.

4.3.5 Scalability

The stateless PostgREST API layer scales horizontally behind a load balancer. Supabase Realtime's distributed channel architecture supports millions of concurrent WebSocket subscribers. Edge Functions scale independently on Deno Deploy infrastructure.

4.3.6 Testability

Each Riverpod provider is unit-tested with riverpod_test using a mock Supabase client. Supabase Edge Functions are unit-tested with Deno's built-in test runner against a local Supabase Docker stack. Flutter integration tests use flutter_test with an in-memory fake repository layer. Target code coverage: ≥ 80% for the domain and data layers.

4.3.7 Interoperability

Supabase exposes a fully standard PostgREST API, meaning any HTTP client future web dashboards, third-party analytics tools, or ERP systems can query the platform. All payment integrations use vendor-standard REST APIs with no proprietary SDKs that would block future gateway switching.

4.3.8 Adaptability

The relational SQL schema allows clean, versioned migrations as requirements grow. Adding a new user role requires only a new role enum value, updated RLS policies, and a Flutter role navigator. The modular architecture allows modules to be enhanced or replaced independently.

5 Design Description

5.1 Composite Viewpoint

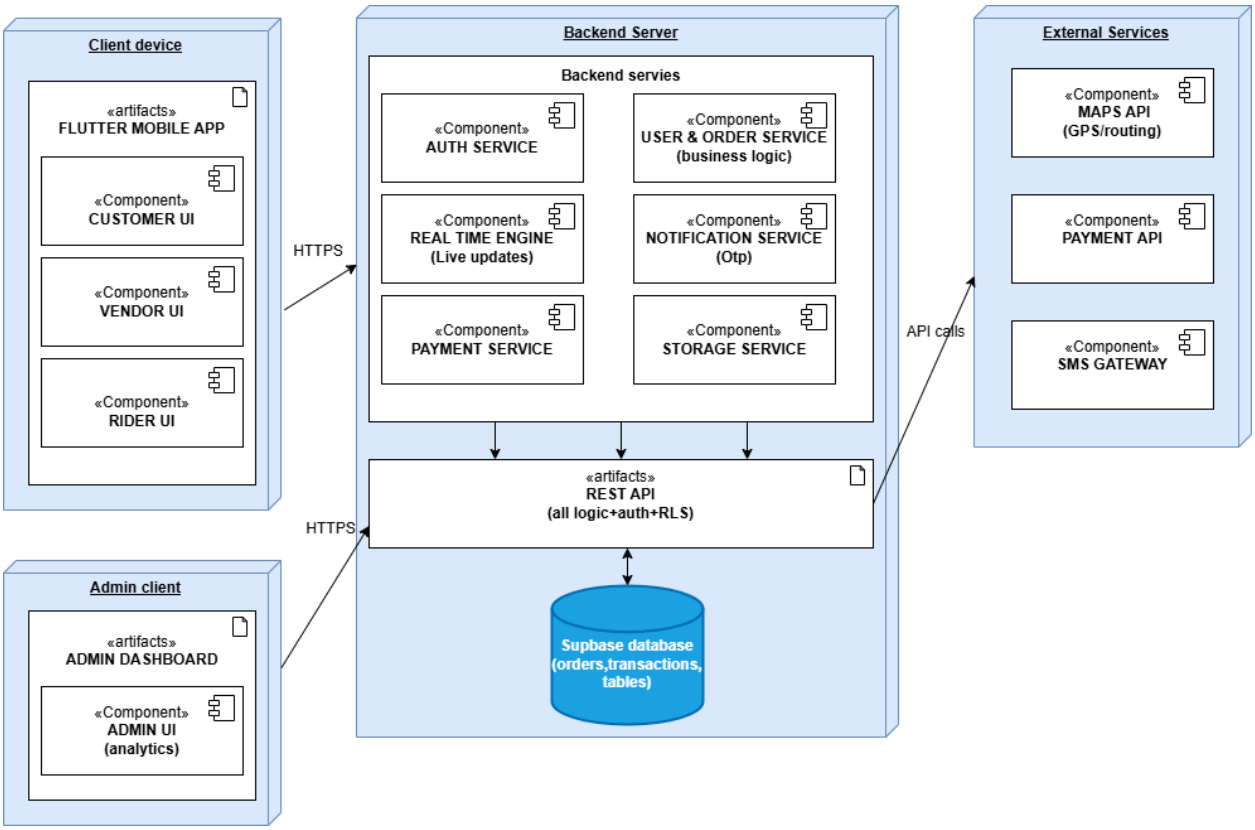


Figure 5.1.1: Deployment Diagram

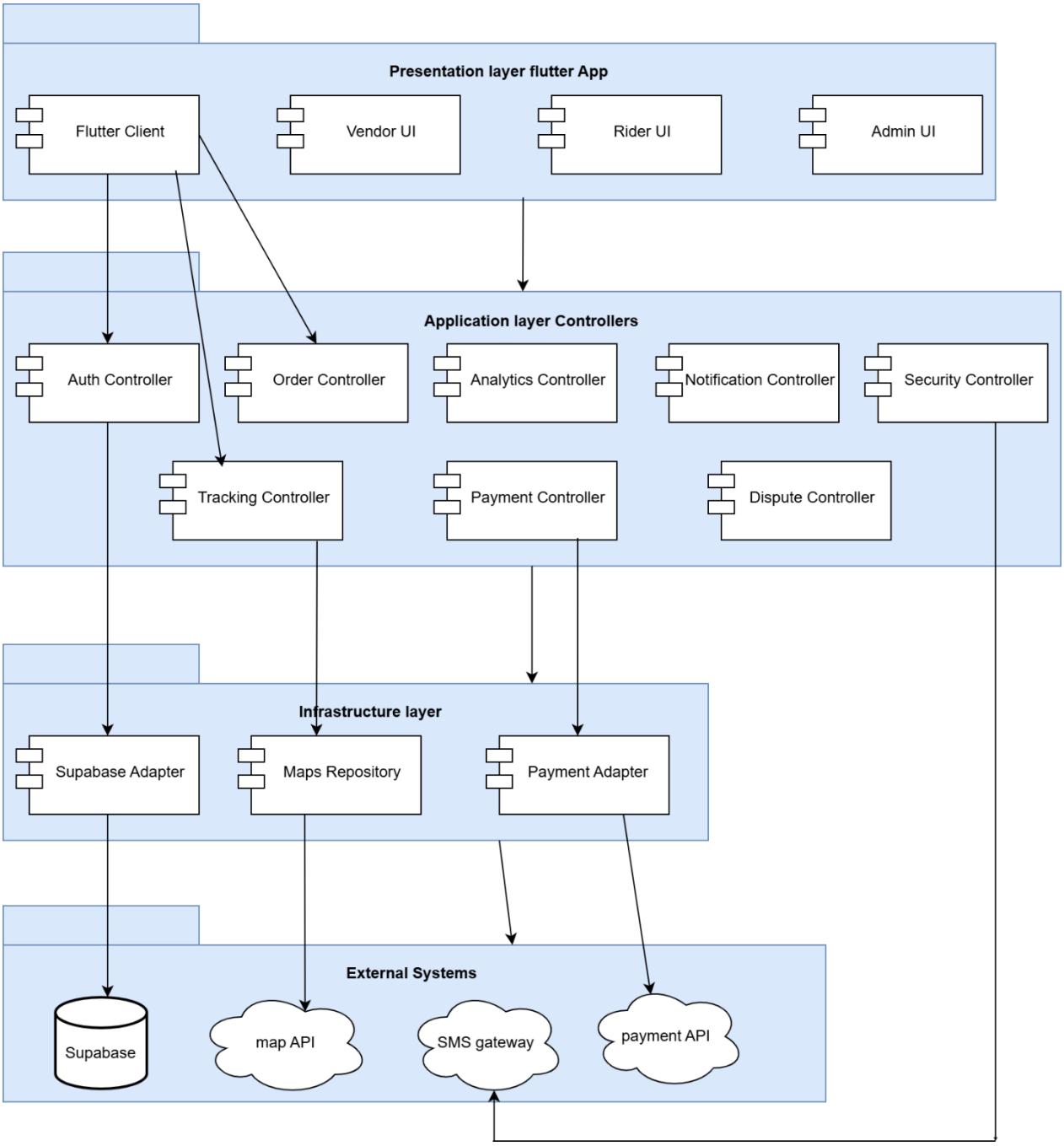


Figure 5.1.2: Package Diagram

5.2 Logical Viewpoint

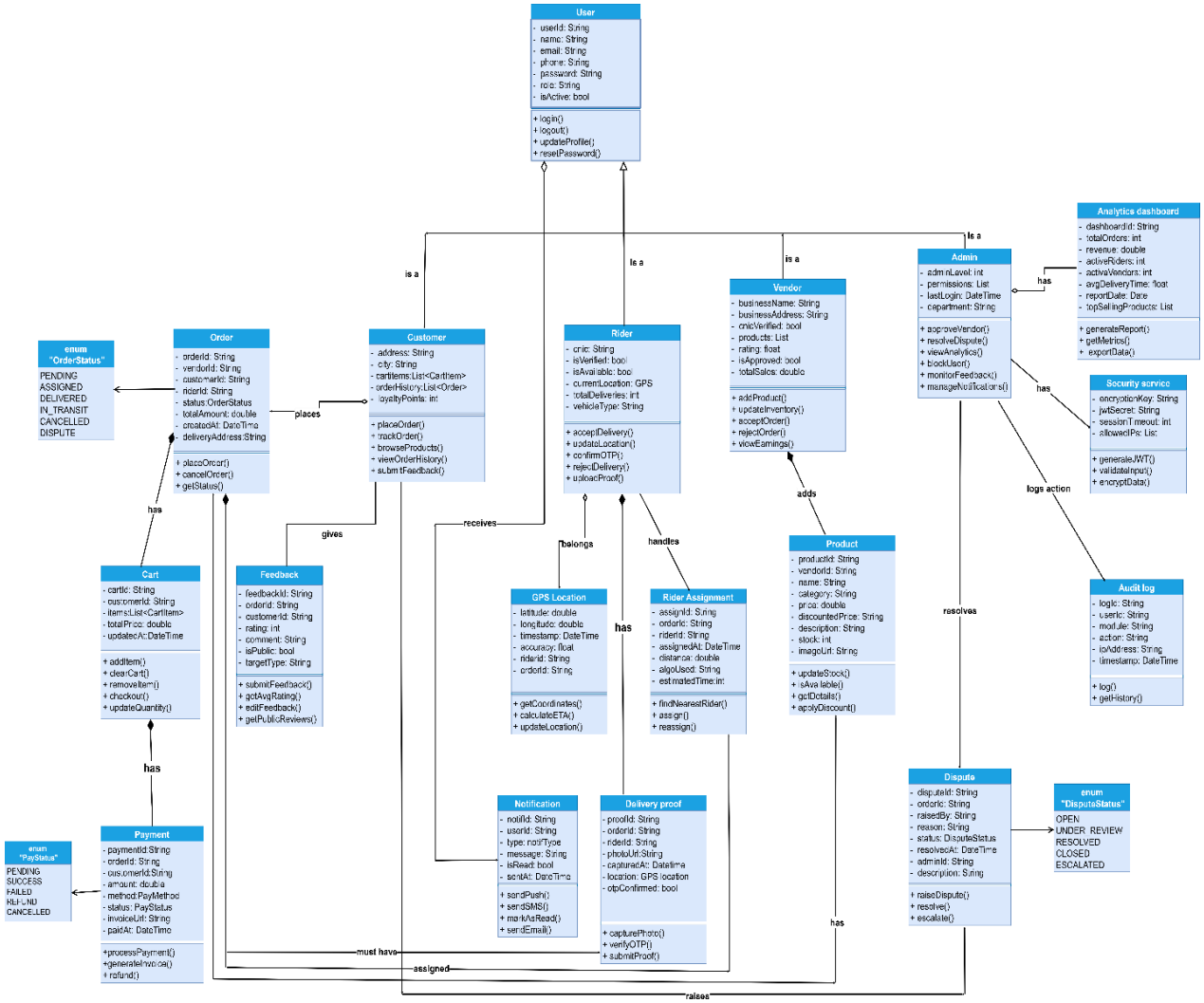


Figure 5.2: Class Diagram

5.3 Information Viewpoint

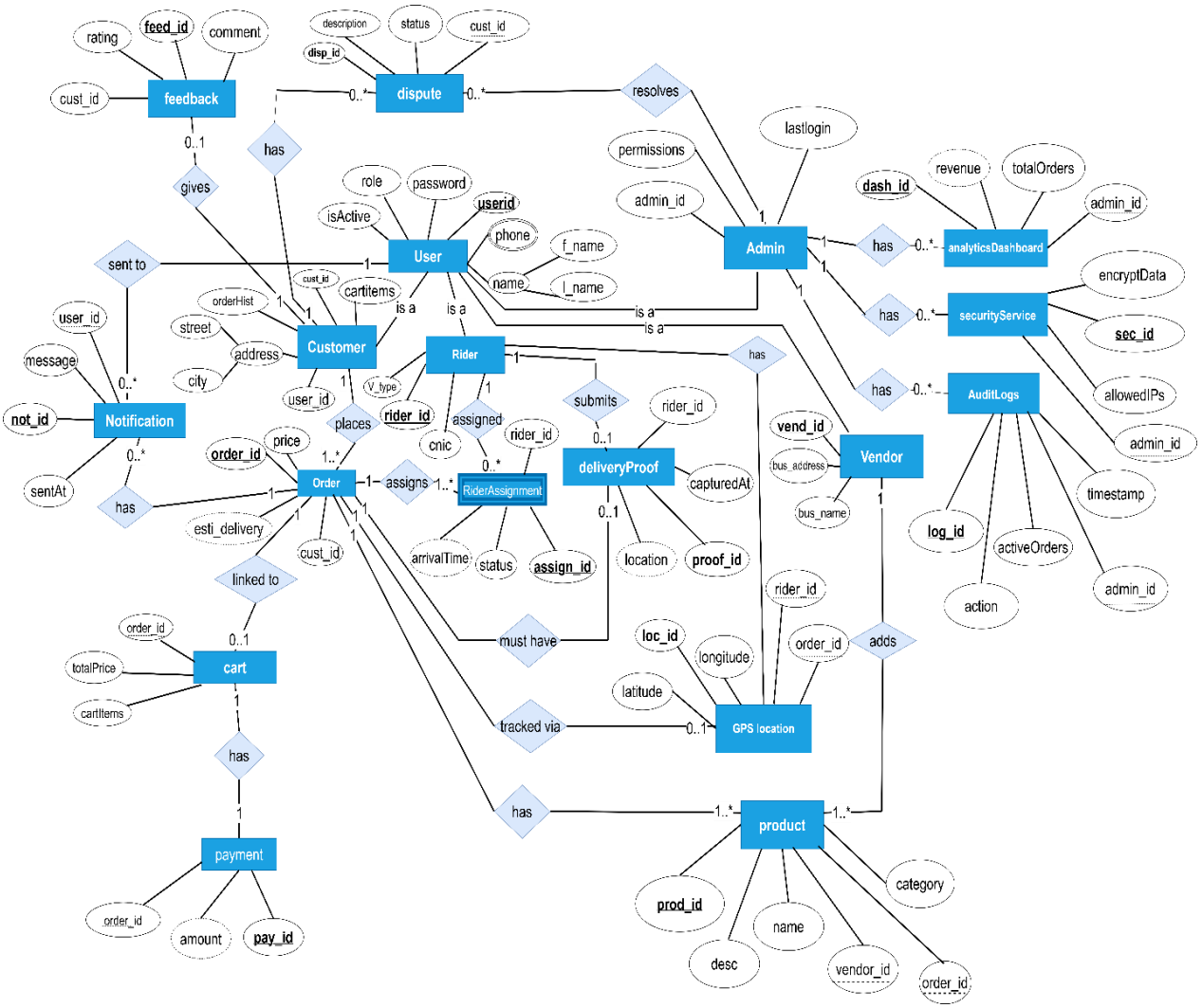


Figure 5.3: Entity Relationship Diagram

5.4 Interaction Viewpoint

5.4.1 Customer View

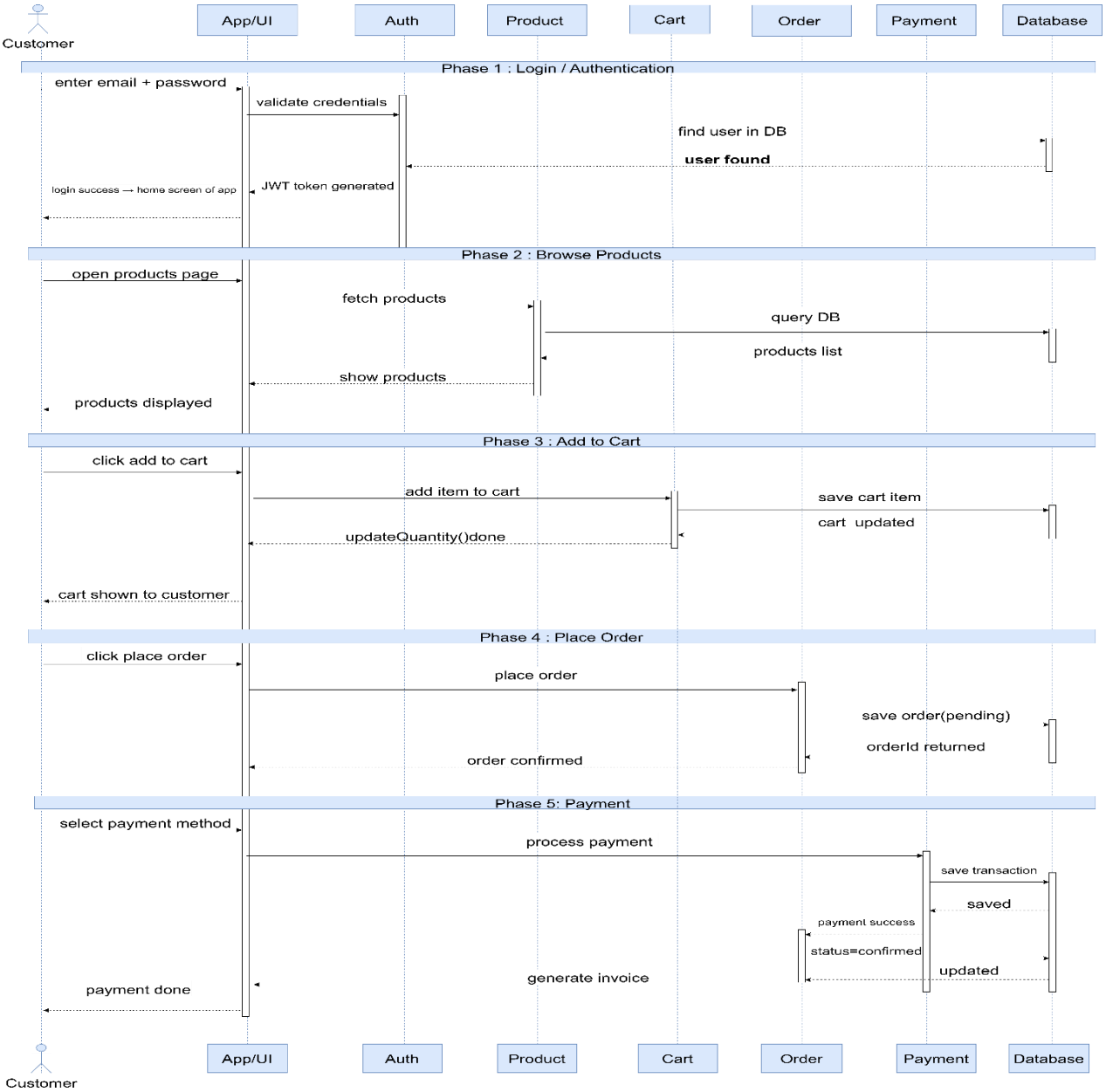


Figure 4.4.1: Customer-View Sequence Diagram

5.4.2 Vendor view

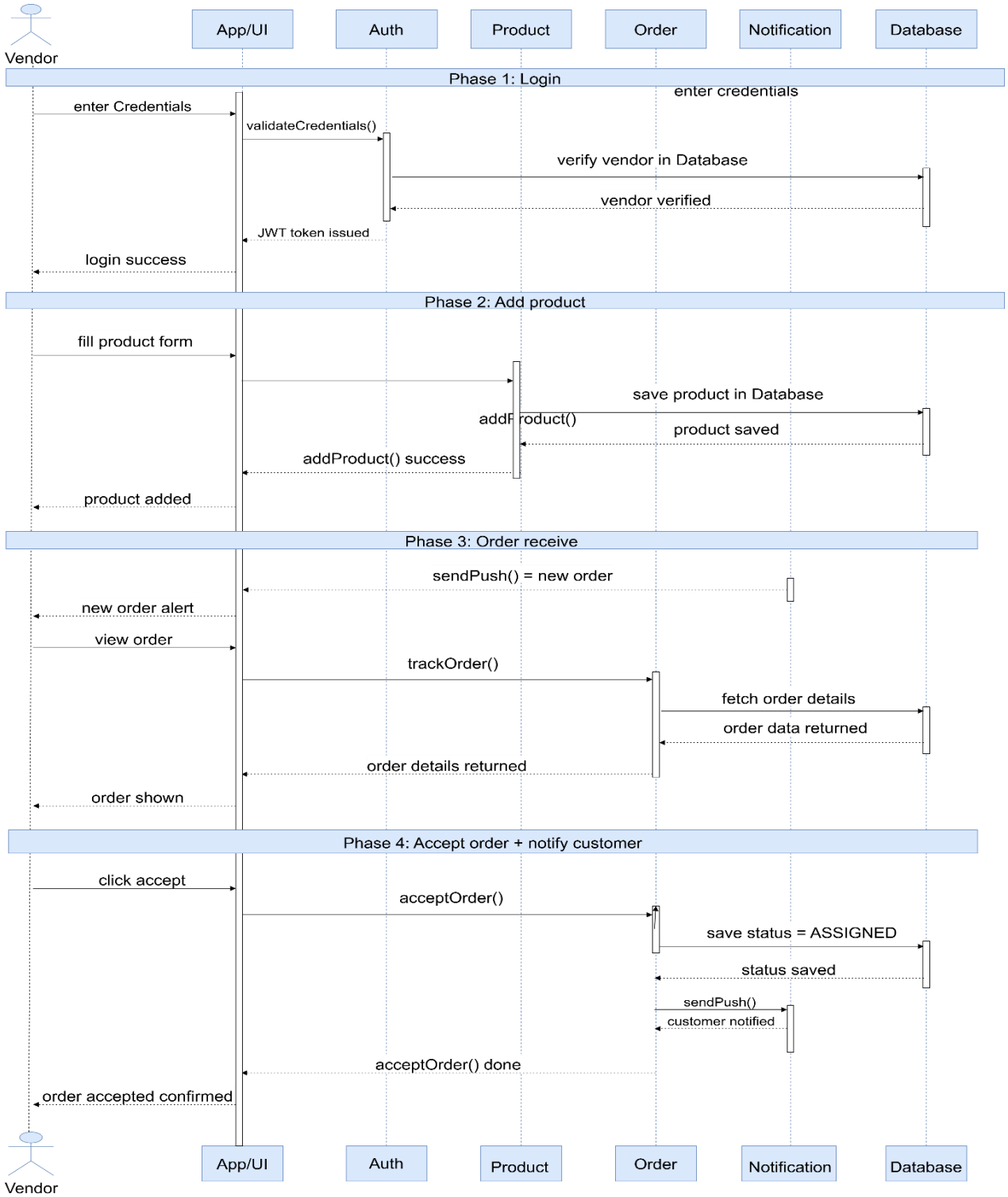


Figure 5.4.2: Vendor-View Sequence Diagram

5.4.3 Rider View

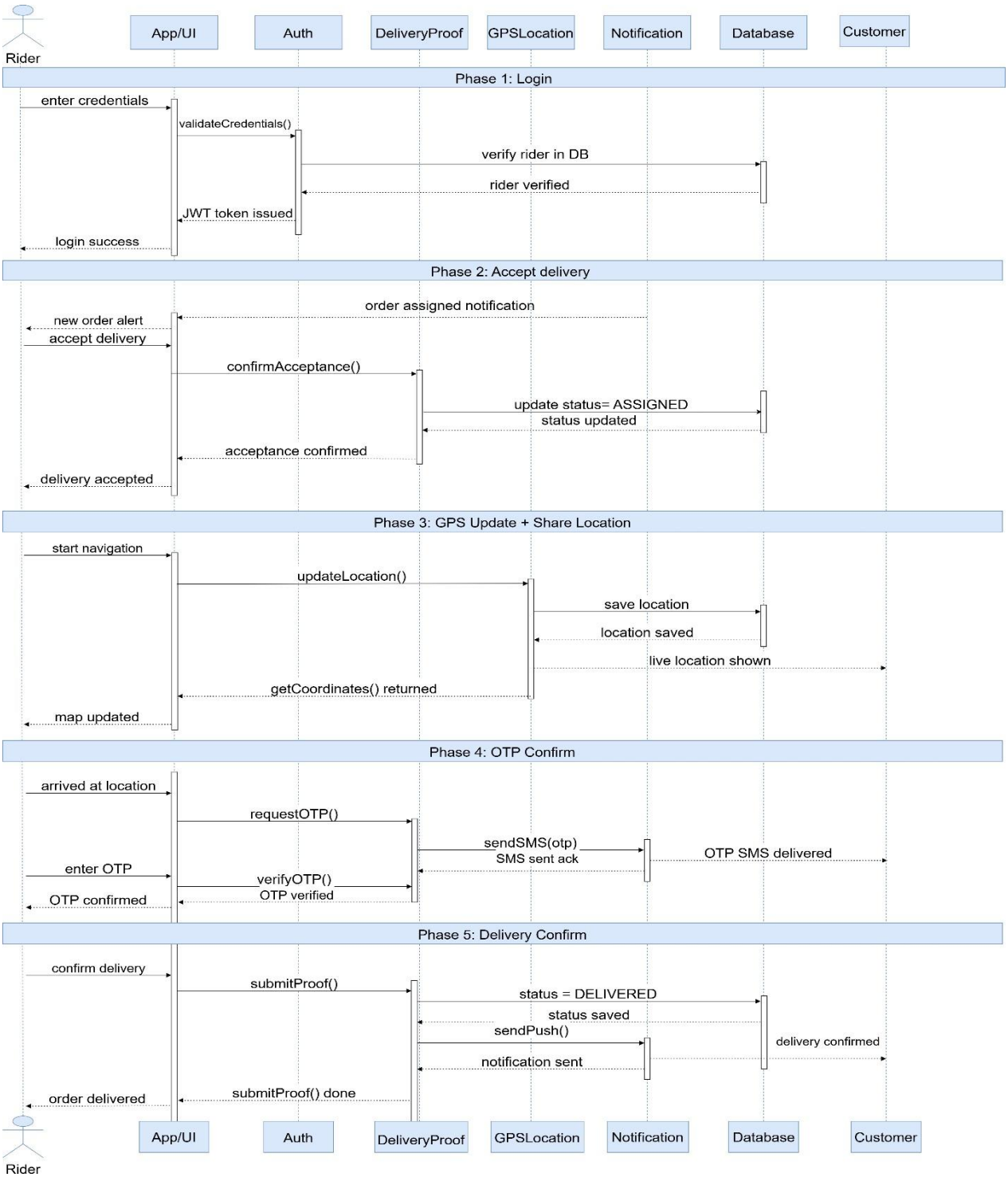


Figure 5.4.3: Rider-View Sequence Diagram

5.4.4 Admin View:

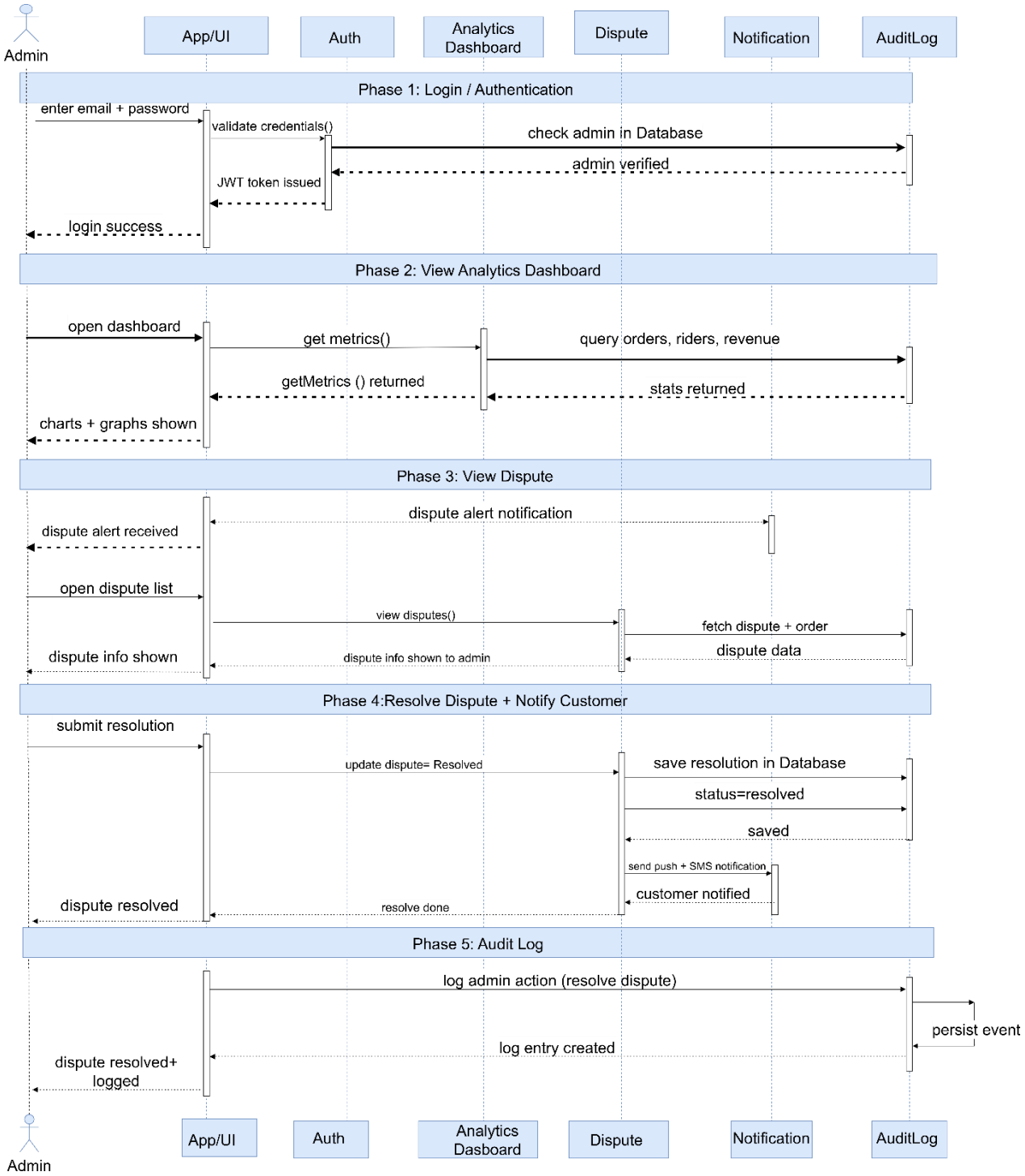


Figure 5.4.4: Admin-View Sequence Diagram

5.5 State Dynamics Viewpoint

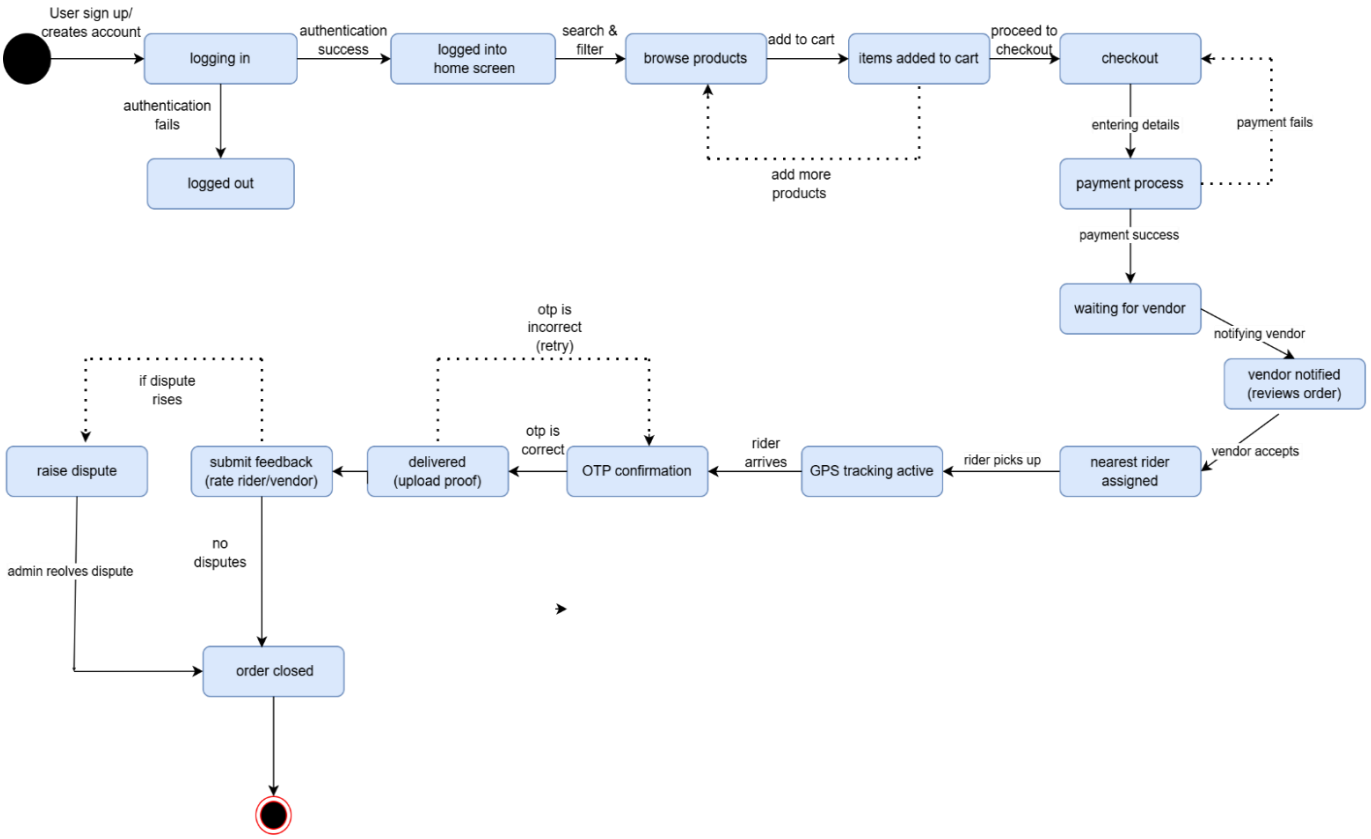


Figure 5.5: State Machine Diagram

5.6 Algorithm Viewpoint

The decision table below formalizes the complete set of system conditions, actions, and outputs across all critical Tez Rider scenarios.

Table 5.6: Algorithm Viewpoint

#	Scenario	Condition	System Action	Output / Notification
1	**Registration**	All fields valid; email unique; role selected	INSERT users; send email verification	'Registration successful. Verify your email.'
2	**Registration**	Email already registered	Reject INSERT	'An account with this email already exists.'

3	**Registration**	Rider — CNIC doc not uploaded	Block registration completion	'Please upload your CNIC document to continue.'
4	**Vendor Approval**	Admin approves vendor	UPDATE vendors SET is_approved=true	'Your account has been approved.'
5	**Vendor Approval**	Admin rejects vendor	UPDATE vendors SET status=REJECTED	'Account not approved. Contact support.'
6	**Rider Approval**	Admin approves rider after CNIC check	UPDATE riders SET is_verified=true	'Your CNIC has been verified. Start accepting jobs.'
7	**Add to Cart**	Product in stock (stock_qty > 0)	INSERT cart_items; decrement reserved stock	Cart badge increments
8	**Add to Cart**	Product out of stock (stock_qty = 0)	Reject; do not insert	'This product is currently out of stock.'
9	**Place Order**	Payment success (HMAC valid)	INSERT orders + order_items; UPDATE payment status=SUCCESS; send invoice email	'Order placed! Awaiting vendor confirmation.'
10	**Place Order**	Payment fails	Rollback order record; log failure	'Payment failed. Please try again.'
11	**Place Order**	Payment webhook HMAC invalid	Reject callback; log security alert	HTTP 401 returned to gateway; alert Admin
12	**Vendor Accept**	Vendor accepts within 10-min timeout	PATCH order status=ACCEPTED; trigger rider assignment	'Vendor confirmed your order.'

13	**Vendor Accept**	Vendor timeout — no response	Auto-cancel; initiate refund	'Order cancelled — vendor unavailable.'
14	**Rider Assignment**	Available verified rider within 5 km	INSERT rider_assignments; notify Rider	'New delivery job assigned.'
15	**Rider Assignment**	No rider within 5 km	Expand search to 10 km; retry once	If still none: notify Admin for manual assignment
16	**Delivery Checklist**	Rider submits complete checklist	UPDATE rider_assignments SET checklist_completed=true	GPS streaming unlocked
17	**OTP Confirmation**	OTP entered correctly	INSERT delivery_proofs; PATCH order status=DELIVERED	'Your order has been delivered.'
18	**OTP Confirmation**	OTP incorrect [attempts < 3]	Decrement attempt counter	'Incorrect OTP. X attempts remaining.'
19	**OTP Confirmation**	3 failed attempts	Expire OTP; flag order for Admin review	'OTP expired. Request a new code.'
20	**Dispute**	Raised within 24 hours with evidence	INSERT disputes; notify Admin	'Dispute submitted. Under review within 48 h.'
21	**Dispute**	Raised after 24-hour window	Reject insertion	'The dispute window for this order has closed.'
22	**Dispute Resolution**	Admin approves refund	Trigger refund API; UPDATE dispute status=RESOLVED	'Refund has been initiated.'
23	**Dispute Resolution**	Admin rejects dispute	UPDATE dispute status=REJECTED	'Dispute rejected. See reason in app.'
24	**Low Stock Alert**	product. stock_qty < threshold	INSERT notification to Vendor	'Low stock alert: [product name].'

25	**Block User**	Admin blocks user	UPDATE users SET status=BLOCKED; revoke JWT session	User receives HTTP 403 on next API call
26	**GPS Update**	Rider sends coordinate whilst assignment active	INSERT gps_locations; broadcast to Realtime channel	Customer map pin updates in real time
27	**GPS Lost**	No GPS update from Rider > 30 seconds	No DB action; Client-side timeout trigger	Customer map: 'Rider location temporarily unavailable.'
28	**Feedback**	Customer submits rating after delivery	INSERT feedback; recompute vendor/rider rating_avg	'Thank you for your feedback!'
29	**Invoice**	Payment SUCCESS event received	Generate PDF invoice; email to Customer	Email: 'Your invoice for Order #X is attached.'
30	**API / DB Error**	Unhandled exception in any endpoint	Log to security_audit_log; return HTTP 500	'Something went wrong. Please try again.'